

CS Cheat Sheet: Data Structures & Algorithms

CS 301 — Midterm Reference

Time Complexity — Common Operations

Structure	Access	Search	Insert	Delete
Array	$O(1)$	$O(n)$	$O(n)$	$O(n)$
Linked List	$O(n)$	$O(n)$	$O(1)$	$O(1)$
Stack / Queue	$O(n)$	$O(n)$	$O(1)$	$O(1)$
Hash Table	-	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg
Binary Search Tree	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$
Balanced BST (AVL)	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$
Heap (Binary)	$O(1)$ min	$O(n)$	$O(\log n)$	$O(\log n)$

Sorting Algorithms

Algorithm	Best	Average	Worst	Space	Stable?
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No

Graph Algorithms

BFS: $O(V+E)$, shortest path (unweighted), uses queue **DFS:** $O(V+E)$, uses stack/recursion, cycle detection, topo sort

Dijkstra: $O((V+E) \log V)$ with min-heap, non-negative weights **Bellman-Ford:** $O(VE)$, handles negative weights

Kruskal / Prim (MST): $O(E \log V)$ **A*:** heuristic-guided shortest path

Recurrence / Master Theorem

$T(n) = aT(n/b) + f(n)$ Compare $f(n)$ to $n^{\log_b a}$:

1) $f(n) = O(n^{\log_b a - \epsilon}) \Rightarrow T(n) = \Theta(n^{\log_b a})$

2) $f(n) = \Theta(n^{\log_b a}) \Rightarrow T(n) = \Theta(n^{\log_b a} \cdot \log n)$

3) $f(n) = \Omega(n^{\log_b a + \epsilon}) \Rightarrow T(n) = \Theta(f(n))$

Big-O Growth (smallest to largest)

$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$